

practical 7.ipynb

E: > Sharvayu data > Malware > Symbiosis Nagpur SIT > 7th SEM > DL LAB > DeepLearning LAB > Practical 7 > practical 7.ip

Generate + Code + Markdown | Run All Clear All Outputs | Outline ...

23070521135 - Sharvayu Zade

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import VGG16, ResNet50, MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

# Set random seeds for reproducible results
tf.random.set_seed(42)
np.random.seed(42)

print('TensorFlow Version:', tf.__version__)
```

... TensorFlow Version: 2.21.0

```
BASE_DIR = 'Dog Breed Classification'
TRAIN_DIR = os.path.join(BASE_DIR, 'train')
VAL_DIR = os.path.join(BASE_DIR, 'val')
TEST_DIR = os.path.join(BASE_DIR, 'test')

# 5 Dog Breed classes selected for classification
CLASSES = ['beagle', 'bernese_mountain_dog', 'golden_retriever', 'pug', 'rottweiler']
```

practical 7.ipynb

E: > Sharvayu data > Malware > Symbiosis Nagpur SIT > 7th SEM > DL LAB > DeepLearning LAB > Practical 7 > practical 7.ip

Generate + Code + Markdown | Run All Clear All Outputs | Outline ...

```
# 5 Dog Breed classes selected for classification
CLASSES = ['beagle', 'bernese_mountain_dog', 'golden_retriever', 'pug', 'rottweiler']
NUM_CLASSES = len(CLASSES)
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

print(f'Selected {NUM_CLASSES} Dog Breed Classes:')
for breed in CLASSES:
    print(f' - {breed}')

# Data Augmentation for training set
train_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

# Rescaling for validation and test sets
val_datagen = ImageDataGenerator(rescale=1.0/255.0)
test_datagen = ImageDataGenerator(rescale=1.0/255.0)

# Flow from directory generators for 5 classes
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=True
)

val_generator = val_datagen.flow_from_directory(
    VAL_DIR,
    target_size=IMG_SIZE,
```

practical 7.ipynb

E: > Sharvayu data > Malware > Symbiosis Nagpur SIT > 7th SEM > DL LAB > DeepLearning LAB > Practical 7 >

Generate Code Markdown Run All Clear All Outputs Outline ...

```
target_size=IMG_SIZE,
batch_size=BATCH_SIZE,
class_mode='categorical',
classes=CLASSES,
shuffle=True
)

val_generator = val_datagen.flow_from_directory(
    VAL_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=False
)

test_generator = test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=False
)
```

... Selected 5 Dog Breed Classes:

- beagle
- bernese_mountain_dog
- golden_retriever
- pug
- rottweiler

Found 363 images belonging to 5 classes.
Found 43 images belonging to 5 classes.
Found 50 images belonging to 5 classes.

practical 7.ipynb

E: > Sharvayu data > Malware > Symbiosis Nagpur SIT > 7th SEM > DL LAB > DeepLearning LAB > Practical 7 > practical 7.ipynb > M4 23070521135

Generate Code Markdown Run All Clear All Outputs Outline ...

```
model_names = ['VGG16', 'ResNet50', 'MobileNetV2']
results = {}

for name in model_names:
    print(f'\n=====')
    print(f'  STARTING TRAINING FOR: {name}')
    print(f'=====')

    # Step 1: Build model & convert 1000-class head to 5-class head
    model, base_model = build_transfer_model(name, num_classes=NUM_CLASSES, input_shape=(224, 224, 3))

    # Step 2: Phase 1 Training - Base Frozen (Feature Extraction)
    print(f'--> Phase 1: Training feature extraction head (Base Frozen)...')
    history_p1 = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=5,
        callbacks=[EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)],
        verbose=1
    )

    # Step 3: Phase 2 Fine-Tuning - Unfreeze Last 10 Layers
    print(f'--> Phase 2: Fine-Tuning (Unfreezing last layers)...')
    model = unfreeze_and_compile(model, base_model, num_unfreeze_layers=10)
    history_p2 = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=3,
        callbacks=[EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)],
        verbose=1
    )

    # Step 4: Evaluate on Test Set
    test_loss, test_acc = model.evaluate(test_generator, verbose=0)
```

```
# Save metrics and epoch trajectory
results[name] = {
    'model': model,
    'test_acc': test_acc,
    'test_loss': test_loss,
    'val_acc': history_p1.history['val_accuracy'] + history_p2.history['val_accuracy'],
    'val_loss': history_p1.history['val_loss'] + history_p2.history['val_loss'],
    'train_acc': history_p1.history['accuracy'] + history_p2.history['accuracy'],
    'train_loss': history_p1.history['loss'] + history_p2.history['loss']
}
```

STARTING TRAINING FOR: VGG16

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.12.0: Phase 1: Training feature extraction head (Base Frozen)...

Epoch 1/5

```
1/12 ----- 2:08 12s/step - accuracy: 0.2500 - loss: 1.7110
2/12 ----- 1:06 7s/step - accuracy: 0.1719 - loss: 1.7696
3/12 ----- 58s 6s/step - accuracy: 0.1771 - loss: 1.7115
4/12 ----- 50s 6s/step - accuracy: 0.2031 - loss: 1.6847
5/12 ----- 44s 6s/step - accuracy: 0.1813 - loss: 1.7048
6/12 ----- 38s 6s/step - accuracy: 0.1979 - loss: 1.6671
7/12 ----- 32s 6s/step - accuracy: 0.1920 - loss: 1.6584
8/12 ----- 25s 6s/step - accuracy: 0.1953 - loss: 1.6671
9/12 ----- 18s 6s/step - accuracy: 0.1910 - loss: 1.6722
```

MODEL COMPARISON TABLE			
Model Architecture	Test Accuracy (%)	Test Loss	Final Val Accuracy (%)
VGG16	78.0	0.8894	65.12
ResNet50	36.0	1.5445	30.23
MobileNetV2	100.0	0.0157	100.00

